27-NumPy Array Operations and Class Discussion

Introduction and Recap

• Continuation from previous class on NumPy array basics

• Focus on advanced array methods, indexing, slicing, and other array operations

Arithmetic Operations on Arrays

• Element-wise addition demonstrated: arrays must have the same shape and size

• New arrays are created with arithmetic operations; original arrays remain unchanged

• In-place operations (`+=`, `-=`, `*=`, `/=`) update an existing array rather than creating a new one

Unary Operations (sum, max, min)

• Functions like `np.sum`, `np.max`, `np.min` work on entire arrays

• Can also compute along specific axes: use of the `axis` parameter

  - `axis=0` performs operation down rows (for each column)

  - `axis=1` performs operation across columns (for each row)

• Examples include finding maximum in each row or column

- Clarification that axis direction (0 for rows, 1 for columns) can sometimes be confusing conceptually and must be noted

Indexing and Slicing

1D Arrays

- Indexing uses positive/negative indices

- Slicing uses standard Python slice notation, can also be done in reverse

2D Arrays

- Rows and columns indexed from 0 upward

- Slicing in 2D: first slice rows, then columns (can combine both in `array[row_slice, column_slice]`)

- Accessing specific row or element: provide row and column indices

- Getting specific columns: use `:` for all rows and specific column index

Advanced and Patternless Indexing

- Accessing arbitrary elements using lists of row and column indices

- Demonstrated how to retrieve non-contiguous elements by passing index arrays/lists

- Showed how to assign new values to selected elements using the same method

Boolean and Condition-based Operations

- Conditional expressions (e.g., `array > 15`) return Boolean arrays of the same shape
- Boolean arrays can be used to select/filter values (e.g., all elements greater than 15)
- Counting conditionally-matching elements can be done with `np.sum(condition)` (since True=1, False=0)
- Explored how to sum only those values meeting a condition, including sum by axis

np.all and np.any

- `np.all(condition)`: returns True if all elements satisfy the condition, else False
- `np.any(condition)`: returns True if any element satisfies the condition, else False

Class Feedback and Practice Session Plans

- Students shared concerns about course pace, overwhelming density of new concepts, and need for practice
- Discussion about whether to have separate sessions for programmers and non-programmers
- Instructor stressed the importance of asking questions and actively engaging
- Suggestions included more revision classes, possible pause after certain modules for practice, and extra sessions as needed

- Plan stated to dedicate several sessions (after finishing Pandas) for practice, with focus on reinforcing core concepts before moving to visualization libraries

Conclusion

- All major indexing, slicing, operations, and Boolean logic concepts in NumPy arrays covered
- Some advanced/iteration concepts postponed for next class
- Students encouraged to review, practice, and communicate their needs for better understanding